Report:

1. S3 Bucket Setup

To begin the ETL workflow, I created an Amazon S3 bucket named holly-lab10-main.
This bucket serves as the main storage location for:

- The original unclean dataset
- The processed and cleaned CSV output

After creating the bucket, I uploaded the raw input file:

- M13_AWS Lab_unclean_data.csv

to the root directory of the bucket.
This file is used as the data source for both the Glue Crawler and Glue ETL job.

2. AWS Glue Crawler Setup

I configured an AWS Glue Crawler to automatically detect the schema of the uploaded CSV file.
Steps performed:

1. Created a new Glue Crawler and selected S3 as the data source.
2. Provided the S3 path of the unclean data file in holly-lab10-main.
3. Selected an IAM role that grants Glue access to S3.
4. Configured the crawler to write metadata to a new Glue Database named: lab10-hollychen
5. Ran the crawler, which successfully generated a table named: holly_lab10_main

The detected schema included the following columns:

- id
- name
- age
- gender
- salary

This table was later used as the input source for PySpark transformations.

3. AWS Glue ETL Job Setup (PySpark)

To transform the dataset, I used an AWS Glue Interactive Notebook.
Steps performed:

1. Opened Glue Studio Notebook and created a new Glue Job.
2. Selected the same IAM role used by the crawler.
3. Loaded the table produced by the crawler:

```
datasource0 = glueContext.create_dynamic_frame.from_catalog(
    database="lab10-hollychen",
    table_name="holly_lab10_main"
)
```

4. Converted the DynamicFrame to a Spark DataFrame to perform transformations:

```
df = datasource0.toDF()
```

5. Displayed the original data using:

```
df.show()
df.printSchema()
```

This allowed me to inspect issues such as missing values, inconsistent formatting, and incorrect data types.

4. PySpark Transformations Performed

To clean and standardize the dataset, I applied a two-stage transformation pipeline, which includes normalization, type conversion, missing-value handling, and rounding.

Stage 1 — Data Preparation (Normalization + Filtering)

I performed the following cleaning steps:

- Removed rows with missing or null id
- Removed rows with missing, null, or empty names
- Normalized missing values in age and salary by converting these to actual NULL:
    - empty strings ""
    - "NaN", "nan"
    - "NULL", "null"
- Converted age and salary into numeric (float) types

These transformations produce a clean, consistent base dataset before computing averages.

Stage 2 — Imputation of Missing Values

After preparation, I computed the average age and salary:

```
avg_age = avg_vals["avg_age"]
avg_salary = avg_vals["avg_salary"]
```

Then I filled the missing numeric values with the corresponding averages:

```
.withColumn("age", F.when(F.col("age").isNull(), F.lit(avg_age)))
.withColumn("salary", F.when(F.col("salary").isNull(), F.lit(avg_salary)))
```

Final Step — Rounding Values

To ensure cleaner, more readable output, both age and salary were rounded to two decimal places:

```
df_cleaned = df_cleaned.withColumn("age", F.round(F.col("age"), 2))
df_cleaned = df_cleaned.withColumn("salary", F.round(F.col("salary"), 2))
```

Verification

I verified the cleaned DataFrame using:

```
df_cleaned.show()
df_cleaned.printSchema()
```

The final dataset contained:

- No missing ids
- No missing names
- Consistent numeric values
- Properly imputed and rounded age/salary values

5. Writing Cleaned Output Back to S3

Finally, I saved the cleaned dataset back into S3, inside the folder cleaned/:

```
output_path = "s3://holly-lab10-main/cleaned/"
df_cleaned.write.format("csv").option("header", True).mode("overwrite").save(output_path)
```

The output folder contains:

- A generated CSV file in the format:

part-00000-c52577fd-06c5-4754-bc68-a82fafedb720-c000.csv

representing the fully cleaned dataset, ready for further analysis or downstream processing.

Screenshots:

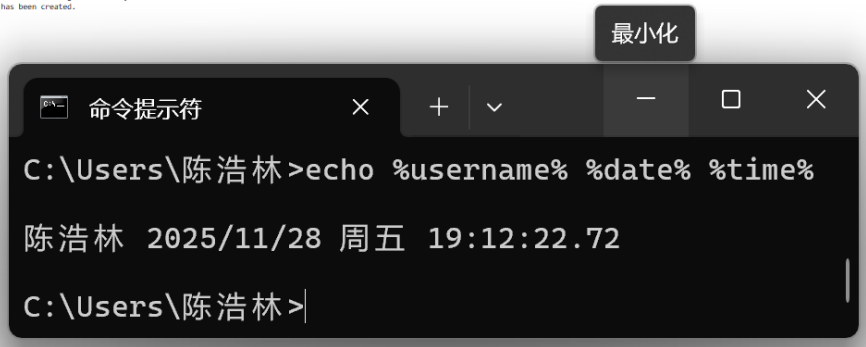
A. PySpark transformation code

```
%idle_timeout 2880
%glue_version 4.0
%worker_type G.1X
%number_of_workers 2

import sys
from awsglue.transforms import *
from awsglue.utils import getResolvedOptions
from pyspark.context import SparkContext
from awsglue.context import GlueContext
from awsglue.dynamicframe import DynamicFrame
from awsglue.job import Job
import pyspark.sql.functions as F

# Initialize Glue Context
sc = SparkContext.getOrCreate()
glueContext = GlueContext(sc)
spark = glueContext.spark_session
job = Job(glueContext)

# ===== Read the table created by the Glue Crawler =====
datasource0 = glueContext.create_dynamic_frame.from_catalog(
    database="lab10-hollychen",
    table_name="holly_lab10_main"
)

df = datasource0.toDF()

print("===== Original Data =====")
df.show()
df.printSchema()

# ===== First stage: prepare data (remove invalid rows, normalize missing values, convert types) =====
df_prepared = (
    df
    # Remove rows with missing or invalid id / name
    .filter(F.col("id").isNotNull())                    # drop rows where id is NULL
    .filter(F.col("name").isNotNull())                  # drop rows where name is NULL
    .filter(F.length(F.trim(F.col("name"))) > 0)        # drop rows where name is empty or whitespace
```

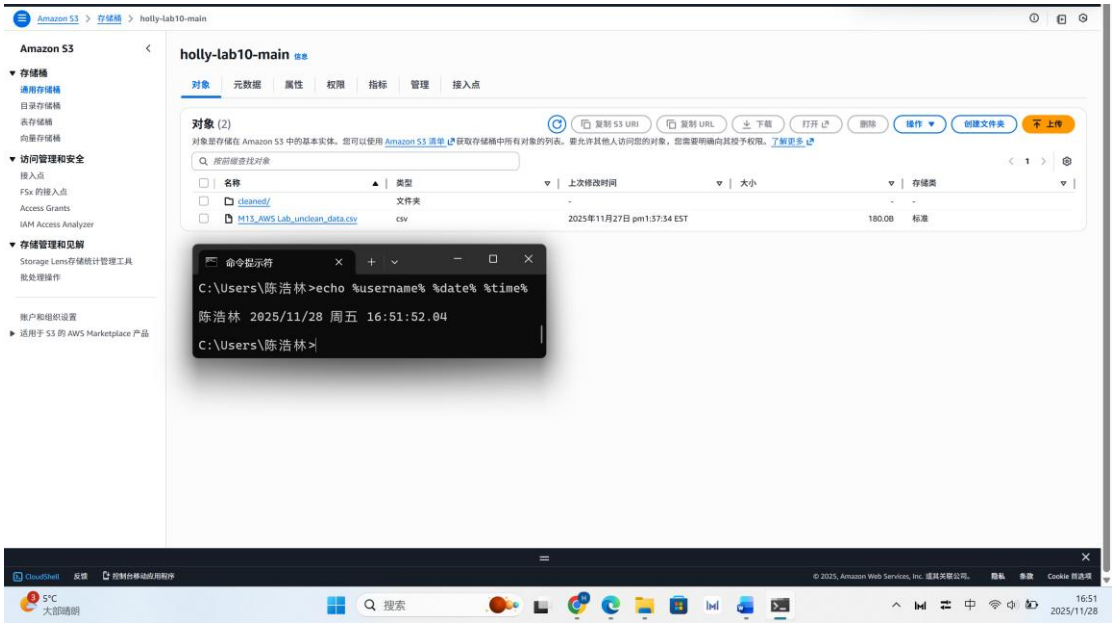
```
    # Normalize missing values in age and salary:
    # convert "", "NaN", "nan", "NULL", "null" to NULL
    .withColumn(
        "age",
        F.when(
            (F.trim(F.col("age")) == "") |
            (F.lower(F.trim(F.col("age"))) == "nan") |
            (F.lower(F.trim(F.col("age"))) == "null"),
            None
        ).otherwise(F.col("age"))
    )
    .withColumn(
        "salary",
        F.when(
            (F.trim(F.col("salary")) == "") |
            (F.lower(F.trim(F.col("salary"))) == "nan") |
            (F.lower(F.trim(F.col("salary"))) == "null"),
            None
        ).otherwise(F.col("salary"))
    )

    # Convert columns to numeric types
    .withColumn("age", F.col("age").cast("float"))
    .withColumn("salary", F.col("salary").cast("float"))
)

print("===== Prepared Data (before filling) =====")
df_prepared.show()
df_prepared.printSchema()

# ===== Compute averages on the prepared dataset =====
avg_vals = df_prepared.select(
    F.avg("age").alias("avg_age"),
    F.avg("salary").alias("avg_salary")
).first()

avg_age = avg_vals["avg_age"]
avg_salary = avg_vals["avg_salary"]

print("Average age:", avg_age)
print("Average salary:", avg_salary)

# ===== Second stage: fill missing numeric values with the corresponding averages =====
```

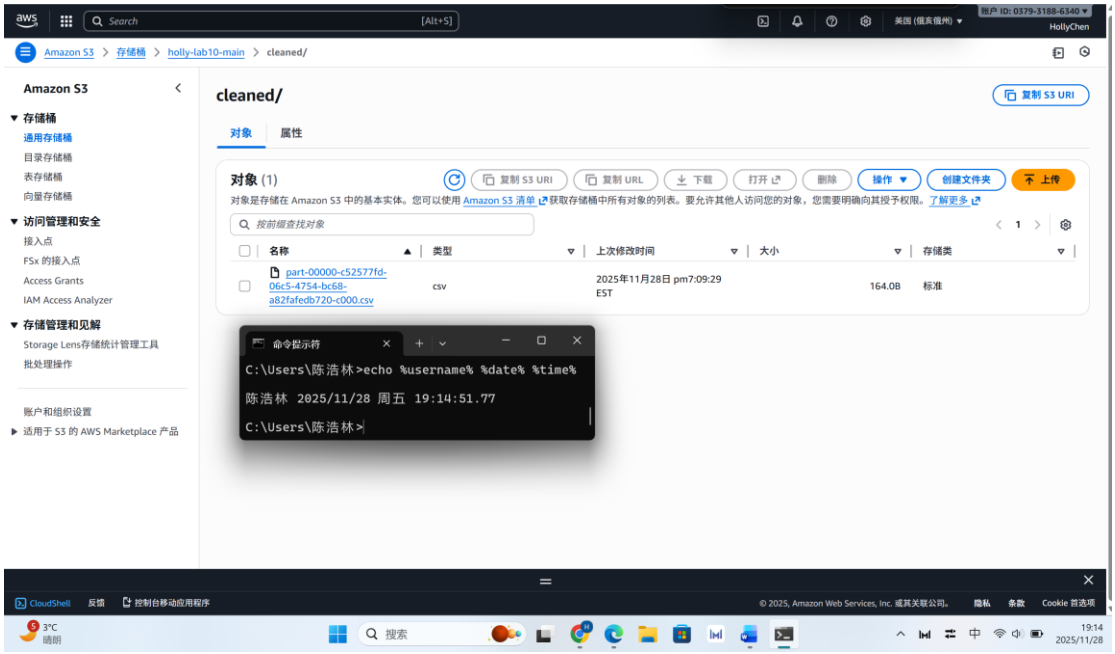
```
df_cleaned = (
    df_prepared
    .withColumn(
        "age",
        F.when(F.col("age").isNull(), F.lit(avg_age)).otherwise(F.col("age"))
    )
    .withColumn(
        "salary",
        F.when(F.col("salary").isNull(), F.lit(avg_salary)).otherwise(F.col("salary"))
    )
)

# ===== Round numeric columns to 2 decimal places =====
df_cleaned = df_cleaned.withColumn("age", F.round(F.col("age"), 2))
df_cleaned = df_cleaned.withColumn("salary", F.round(F.col("salary"), 2))

print("===== Cleaned Data =====")
df_cleaned.show()
df_cleaned.printSchema()

# ===== Write cleaned data back to S3 =====
output_path = "s3://holly-lab10-main/cleaned/"

(df_cleaned
    .write
    .format("csv")
    .option("header", True)
    .mode("overwrite")
    .save(output_path)
)

print("===== Cleaned data has been written to: {} =====".format(output_path))

job.commit()
```

```
Welcome to the Glue Interactive Sessions Kernel
For more information on available magic commands, please type %%help in any new cell.

Please view our Getting Started page to access the most up-to-date information on the Interactive Sessions kernel: https://docs.aws.amazon.com/glue/latest/dg/interactive-sessions.html
Installed kernel version: 1.0.7
Current idle_timeout is None minutes.
idle_timeout has been set to 2880 minutes.
Setting Glue version to: 4.0
Previous worker type: None
Setting new worker type to: G.1X
Previous number of workers: None
Setting new number of workers to: 2
Trying to create a Glue session for the kernel.
Session Type: glueetl
Worker Type: G.1X
Number of Workers: 2
Idle Timeout: 2880
Session ID: 93c38a5d-29fe-4481-a660-855f7a18d1b8
Applying the following default arguments:
--glue_kernel_version 1.0.7
--enable-glue-datacatalog true
Waiting for session 93c38a5d-29fe-4481-a660-855f7a18d1b8 to get into ready status...
Session 93c38a5d-29fe-4481-a660-855f7a18d1b8 has been created.

===== Original Data =====
+----+-----------+---+------+------+
| id|       name|age|gender|salary|
+----+-----------+---+------+------+
|  1|   John Doe| 35|     M| 50000|
|  2|   Jane Doe|   |     F|   NaN|
|  3|           | 45|     M| 55000|
|  4|Alice Brown| 29|     F| 60000|
|null|Chris Green| 40|     M| 70000|
|  6|  Bob Smith|NaN|     M| 50000|
|  7| Mary Black| 32|     F|      |
+----+-----------+---+------+------+

root
 |-- id: long (nullable = true)
 |-- name: string (nullable = true)
 |-- age: string (nullable = true)
 |-- gender: string (nullable = true)
 |-- salary: string (nullable = true)

===== Prepared Data (before filling) =====
+---+-----------+----+------+-------+
| id|       name| age|gender| salary|
+---+-----------+----+------+-------+
|  1|   John Doe|35.0|     M|50000.0|
|  2|   Jane Doe|null|     F|   null|
|  4|Alice Brown|29.0|     F|60000.0|
|  6|  Bob Smith|null|     M|50000.0|
|  7| Mary Black|32.0|     F|   null|
+---+-----------+----+------+-------+

root
 |-- id: long (nullable = true)
 |-- name: string (nullable = true)
 |-- age: float (nullable = true)
 |-- gender: string (nullable = true)
 |-- salary: float (nullable = true)

Average age: 32.0
Average salary: 53333.333333333336
===== Cleaned Data =====
+---+-----------+----+------+--------+
| id|       name| age|gender|  salary|
+---+-----------+----+------+--------+
|  1|   John Doe|35.0|     M| 50000.0|
|  2|   Jane Doe|32.0|     F|53333.33|
|  4|Alice Brown|29.0|     F| 60000.0|
|  6|  Bob Smith|32.0|     M| 50000.0|
|  7| Mary Black|32.0|     F|53333.33|
+---+-----------+----+------+--------+

root
 |-- id: long (nullable = true)
 |-- name: string (nullable = true)
 |-- age: double (nullable = true)
 |-- gender: string (nullable = true)
 |-- salary: double (nullable = true)

===== Cleaned data has been written to: s3://holly-lab10-main/cleaned/ =====
/opt/amazon/spark/python/lib/pyspark.zip/pyspark/sql/dataframe.py:127: UserWarning: DataFrame constructor is internal. Do not directly use it.
```



B. S3 bucket




aws
Search
[Alt+S]
美国 (俄亥俄州)
HollyChen
Amazon S3 > 存储桶 > holly-lab10-main > cleaned/
Amazon S3
存储桶
通用存储桶
目录存储桶
表存储桶
向量存储桶
访问管理和安全
接入点
FSx 的接入点
Access Grants
IAM Access Analyzer
存储管理和见解
Storage Lens存储统计管理工具
批处理操作
账户和组织设置
适用于 S3 的 AWS Marketplace 产品
cleaned/
复制 S3 URI
对象
属性
对象 (1)
复制 S3 URI
复制 URL
下载
打开
删除
操作
创建文件夹
上传
对象是存储在 Amazon S3 中的基本实体。您可以使用 Amazon S3 清单 获取存储桶中所有对象的列表。要允许其他人访问您的对象，您需要明确向其授予权限。了解更多
按前缀查找对象
名称
类型
上次修改时间
大小
存储类
part-00000-c52577fd-06c5-4754-bc68-a82fafedb720-c000.csv
csv
2025年11月28日 pm7:09:29 EST
164.0B
标准
命令提示符
C:\Users\陈浩林>echo %username% %date% %time%
陈浩林 2025/11/28 周五 19:14:51.77
C:\Users\陈浩林>
CloudShell
反馈
控制台移动应用程序
© 2025, Amazon Web Services, Inc. 或其关联公司。
隐私
条款
Cookie 首选项
3°C
晴朗
搜索
19:14
2025/11/28